

SISTEMA ESCOLAR

Relatório Detalhado de Desenvolvimento

Disciplina	Programação Orientada a Objetos
Integrantes	Alex de Souza Oliveira Eduarda Paes de Barros Antonio Vinicius de Vasconcelos Paixão Filipe de Carvalho Teotônio
Tecnologias	Java 17, Spring Boot 4.1, Spring Data JPA, PostgreSQL, HTML, CSS e JavaScript
Período	2026.1
Data	Setembro de 2026

Documento elaborado a partir do código-fonte, testes, diagrama de classes, coleção de API e histórico Git do projeto.

1. Apresentação do Problema

Instituições de ensino precisam manter informações coerentes sobre pessoas, componentes curriculares, turmas, matrículas e avaliações. Quando esses dados ficam distribuídos em documentos ou registros independentes, surgem dificuldades para localizar informações, evitar duplicidades e acompanhar a situação acadêmica de cada aluno. O Sistema Escolar foi desenvolvido como uma solução acadêmica para centralizar esse conjunto mínimo de dados e, ao mesmo tempo, demonstrar a aplicação prática dos conceitos de orientação a objetos.

O sistema oferece uma interface web integrada a uma API REST. A interface permite realizar cadastros e consultar registros; a API organiza as operações de criação, leitura, atualização e exclusão. O PostgreSQL mantém os dados de forma relacional, preservando vínculos como professor responsável pela turma, disciplina ofertada, aluno matriculado e notas associadas à matrícula.

Objetivo geral

Desenvolver uma aplicação escolar organizada em camadas, com domínio orientado a objetos, persistência relacional, validação de dados, tratamento uniforme de erros e testes automatizados.

Objetivos específicos

- Representar pessoas por uma classe abstrata e especializações para aluno e professor.
- Cadastrar disciplinas e formar turmas com professor responsável.
- Matricular um aluno em uma turma sem permitir a repetição do mesmo vínculo.
- Registrar notas entre zero e dez para uma matrícula existente.
- Disponibilizar os recursos por endpoints REST e por uma interface web responsiva.
- Comprovar o comportamento por testes JPA, testes MockMvc e requisições de demonstração.

Públicos e uso esperado

A solução foi pensada para um contexto acadêmico simplificado. Um usuário administrativo cadastra as informações básicas; docentes ficam associados às turmas; alunos recebem matrículas e notas. O projeto não pretende substituir um sistema acadêmico institucional, mas fornece uma base extensível para incorporar autenticação, frequência, períodos letivos e relatórios.

Resultado funcional: seis recursos de negócio expostos pela API e manipulados pela página inicial do sistema.

2. Modelo de Classes

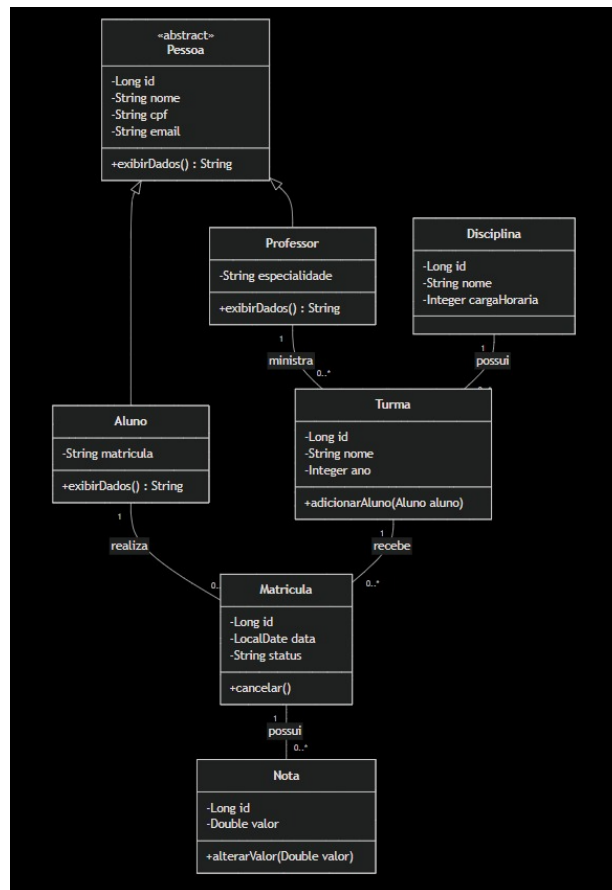


Figura 1 - Modelo de classes do domínio escolar.

O núcleo do modelo inicia em **Pessoa**, uma classe abstrata com `id`, `nome`, `CPF`, `e-mail` e o contrato `exibirDados()`. **Aluno** acrescenta o código de matrícula institucional, enquanto **Professor** acrescenta sua especialidade. Essa hierarquia reduz duplicação e permite tratar ambos como pessoas sem perder seus comportamentos particulares.

Disciplina descreve o componente curricular e sua carga horária. **Turma** representa uma oferta da disciplina em determinado ano e referencia um professor responsável. A associação muitos-para-um permite que um professor e uma disciplina apareçam em várias turmas.

Matrícula materializa a participação de um aluno em uma turma, registrando data e status. A combinação de aluno e turma possui restrição de unicidade. **Nota** referencia uma matrícula e armazena um valor entre zero e dez. Assim, cada avaliação pode ser rastreada até o aluno, a turma, o professor e a disciplina.

2.1 Decisões de Modelagem e Conceitos de POO

Conceito	Aplicação	Evidência no código
Abstração	Pessoa reúne características comuns sem representar um objeto diretamente instanciável.	abstract class Pessoa
Herança	Aluno e Professor reutilizam estado e comportamento de Pessoa.	extends Pessoa
Polimorfismo	Cada subtipo fornece uma versão específica de <code>exibirDados()</code> .	@Override
Encapsulamento	Atributos privados são acessados por métodos e construtores.	private, getters e setters
Associação	As entidades preservam os vínculos acadêmicos por referências tipadas.	@ManyToOne e @JoinColumn
Interfaces	Repositórios definem contratos de persistência e herdam operações prontas.	JpaRepository

Mapeamento objeto-relacional

Pessoa usa a estratégia de herança JPA JOINED. A tabela pessoa guarda os dados comuns, e as tabelas aluno e professor armazenam somente os atributos especializados. A solução favorece normalização e mantém uma identidade única para cada pessoa. As demais entidades usam chaves estrangeiras explícitas, permitindo que o PostgreSQL proteja a integridade referencial.

Regras de integridade

- CPF e e-mail são únicos para todas as pessoas; o código do aluno também é único.
- Turma exige disciplina e professor válidos.
- Matrícula exige aluno e turma existentes; a dupla aluno-turma não pode se repetir.
- Nota exige matrícula existente e valor de 0,0 a 10,0.
- Exclusões que rompem vínculos podem ser rejeitadas e convertidas em resposta HTTP 409.

DTOs e entidades

Os DTOs de entrada concentram validações de formato e obrigatoriedade, enquanto os DTOs de saída controlam o JSON devolvido. Essa separação evita acoplar clientes ao formato interno das entidades e reduz o risco de serializar relações de maneira recursiva.

Decisões e alternativas consideradas

A matrícula foi representada como entidade própria, e não como uma lista direta em Aluno ou Turma, porque possui dados e ciclo de vida próprios. A Nota referencia Matrícula para conservar o contexto acadêmico do lançamento. O professor foi incluído diretamente em Turma porque o escopo atual trabalha com um responsável principal; uma futura modelagem poderia adotar uma entidade de alocação para vários docentes.

3. Arquitetura da Solução

A aplicação segue uma arquitetura em camadas, organizada por responsabilidade. O navegador carrega os arquivos estáticos da interface e utiliza fetch para enviar JSON à API. Os controllers recebem a requisição, acionam a validação dos DTOs e delegam o processamento aos serviços. Os serviços coordenam regras de fluxo e transações. Os repositórios Spring Data JPA executam as operações sobre o PostgreSQL.

Fluxo principal de uma requisição

Etapa	Componente	Responsabilidade
1	Interface web / Postman	Monta a requisição HTTP e envia JSON.
2	Controller + DTO	Interpreta o corpo, valida campos e define o status de resposta.
3	Service	Busca dependências, aplica o fluxo e controla a transação.
4	Repository	Converte operações de domínio em acesso JPA.
5	Hibernate + PostgreSQL	Executa SQL, cria chaves e preserva integridade.
6	ControllerAdvice	Converte exceções em respostas 400, 404 ou 409.

Organização dos componentes

- negocio.basica: entidades Pessoa, Aluno, Professor, Disciplina, Turma, Matricula e Nota.
- negocio.servico: serviços CRUD e composição dos relacionamentos.
- dados: sete interfaces de repositório.
- comunicacao.dto: contratos de entrada e saída.
- comunicacao: controllers REST e tratamento global de exceções.
- resources/static: index.html, style.css e app.js.

Tecnologias utilizadas

Java 17 fornece a base da aplicação. Spring Boot 4.1 inicializa o contexto e o Tomcat embarcado. Spring Web MVC implementa os endpoints; Bean Validation verifica os DTOs; Spring Data JPA e Hibernate realizam o mapeamento relacional. PostgreSQL 18.4 foi usado nos testes locais. Maven gerencia dependências e execução. JUnit 5, Mockito e MockMvc formam a estratégia de testes.

Execução e configuração

EscolaApplication é o ponto de entrada. O Spring escaneia componentes a partir do pacote raiz, registra sete repositórios e publica o Tomcat na porta 8080. O application.yml configura a conexão JDBC e o comportamento do Hibernate. No ambiente atual, ddl-auto: create reconstrói as tabelas; esse modo simplifica o desenvolvimento, mas deve ser substituído antes de uso permanente.

Transações e consistência

Operações de escrita nos serviços usam @Transactional. As consultas utilizam readOnly quando apropriado. Ao criar turma, matrícula ou nota, o serviço busca primeiro as entidades referenciadas; um identificador inválido produz RecursoNaoEncontradoException antes da gravação.

O servidor entrega a interface e a API na mesma origem, evitando configuração adicional de CORS no cenário local.

3.1 API REST, Validação e Interface Web

Recurso	Caminho base	Dados centrais
Aluno	/api/alunos	nome, CPF, e-mail, matrícula
Professor	/api/professores	nome, CPF, e-mail, especialidade
Disciplina	/api/disciplinas	nome e carga horária
Turma	/api/turmas	nome, ano, disciplinaId e professorId
Matrícula	/api/matriculas	data, status, alunoId e turmaId
Nota	/api/notas	valor e matriculaId

Operações e códigos HTTP

Método	Operação	Sucesso	Possíveis falhas
GET	listar ou buscar por id	200	404 ao buscar id inexistente
POST	criar	201 + Location	400, 404 ou 409
PUT	atualizar	200	400, 404 ou 409
DELETE	excluir	204	404 ou 409

Validação e mensagens de erro

Campos obrigatórios utilizam @NotBlank ou @NotNull; identificadores usam @Positive; e-mail usa @Email; carga horária precisa ser positiva; data da matrícula não pode estar no futuro; nota fica limitada de zero a dez. Quando uma validação falha, o ControllerAdvice retorna status 400 e um mapa com mensagens por campo. Recursos ausentes retornam 404. Violações de unicidade ou integridade referencial retornam 409.

Interface web

A página inicial apresenta abas para os seis recursos, constrói os formulários dinamicamente e consome a API com fetch. O usuário pode cadastrar, atualizar a listagem e excluir registros. O layout adapta-se a telas menores e apresenta mensagens de sucesso ou erro. A edição de registros permanece como melhoria futura, pois atualmente o PUT está disponível na API, mas não possui formulário no site.

3.2 Testes e Verificação da Solução

A verificação combina testes de persistência, testes de controllers e execução real da aplicação. Essa combinação reduz o risco de validar somente partes isoladas: os testes JPA conferem o mapeamento com PostgreSQL, enquanto MockMvc verifica status HTTP, estrutura JSON, validação e tratamento de exceções.

Resumo da execução

Indicador	Resultado observado
Testes executados	15
Falhas / erros / ignorados	0 / 0 / 0
Repositórios encontrados	7
Banco utilizado	PostgreSQL 18.4 - esquema escola/public
Interface web	HTTP 200 e página inicial reconhecida pelo Spring
Novos endpoints	/api/matriculas e /api/notas retornaram HTTP 200

Cenários automatizados

- Persistência e consulta de aluno, professor, pessoa, disciplina e turma.
- Persistência da turma com disciplina e professor responsáveis.
- Listagem e criação por controllers, incluindo resposta 201.
- Rejeição de aluno inválido, disciplina com carga zero e turma sem vínculos.
- Resposta 404 para recurso inexistente e 204 para exclusão.
- Criação de matrícula e validação de nota acima de dez.

Roteiro de teste ponta a ponta

Ordem	Ação	Evidência esperada
1	Criar disciplina e professor	Respostas 201 com ids
2	Criar aluno e turma	Turma devolve professor e disciplina
3	Realizar matrícula	Matrícula devolve aluno e turma
4	Registrar nota	Nota identifica matrícula, aluno e turma
5	Enviar nota 11	Resposta 400 com erro no campo valor

Limites da verificação

Os testes atuais comprovam o funcionamento essencial, mas compartilham o PostgreSQL local nos testes de repositório. Uma evolução recomendada é criar um perfil de teste isolado com banco efêmero e dados independentes. Também convém acrescentar cenários de atualização, exclusão com vínculos, matrícula duplicada e várias notas para a mesma matrícula.

Critério de aceite da demonstração

O fluxo é considerado aprovado quando todos os cadastros retornam os códigos esperados, as listagens exibem os relacionamentos e a tentativa inválida retorna mensagem por campo. O navegador e o Postman devem apresentar os mesmos resultados porque utilizam a mesma API.

A coleção Sistema-Escolar-Completo.postman_collection.json reúne esse fluxo para demonstração manual.

4. Dificuldades Encontradas

Configuração do ambiente

O projeto dependia do JDK 17, Maven, PostgreSQL e das credenciais corretas. A conexão foi confirmada pelo HikariCP e pelas informações do banco impressas no log de inicialização.

Porta 8080 ocupada

Uma execução anterior continuou ativa e impediu o Tomcat de iniciar. A análise do log isolou a causa; o processo antigo foi encerrado antes de uma nova execução.

Requisição POST retornando 400

O cliente enviava o corpo raw sem Content-Type application/json. A configuração manual do cabeçalho permitiu ao Spring converter o JSON para o DTO.

Evolução dos relacionamentos

A inclusão de professor em turma alterou construtores, DTOs, serviços e testes. A solução atualizou todas as camadas em conjunto para manter o contrato consistente.

Matrícula duplicada

Permitir o mesmo aluno duas vezes na mesma turma produziria inconsistência. Uma restrição única no banco e uma verificação no serviço passaram a bloquear a duplicidade.

Validação de notas

O intervalo acadêmico precisava ser aplicado antes da persistência. O DTO usa limites decimal mínimo e máximo, e o teste MockMvc comprova a rejeição de nota 11.

Persistência destrutiva

ddl-auto: create facilita a reconstrução durante o desenvolvimento, porém apaga todos os dados a cada inicialização. A equipe identificou a necessidade de migrar para Flyway ou Liquibase.

Rastreabilidade no Git

Parte da camada REST e das extensões atuais ainda aparece como alteração local. A entrega final precisa de commits ou PRs separados, com autoria e mensagens relacionadas às responsabilidades declaradas.

Aprendizado técnico

Os problemas reforçaram a importância de interpretar logs antes de alterar o código, validar contratos HTTP de maneira automatizada, manter entidades e DTOs separados e tratar restrições de banco como parte das regras de negócio.

5. Matriz de Responsabilidades

A matriz relaciona os integrantes aos artefatos observados ou às contribuições que precisam ser formalizadas. O histórico Git disponível comprova commits de Eduarda e Alex. As responsabilidades de Antonio e Filipe devem ser associadas a commits ou PRs próprios antes da entrega para atender integralmente ao critério de evidência.

Membro	Funcionalidade / requisito	Artefato	Evidência Git
Eduarda Paes de Barros	Modelagem inicial; Disciplina, Turma, Aluno e Professor; testes e documentação.	Entidades, repositórios, testes JPA e README.	6f60334, f59d146, 871faec, 96aa115, 0b667d6 e 3baccd4
Alex de Souza Oliveira	Validações do domínio e apoio ao contrato de entrada.	Bean Validation e restrições de persistência.	5d280a0; registrar novos commits se houver
Antonio Vinicius de Vasconcelos Paixão	Matrícula, Nota, associação Professor-Turma e interface web.	Entidades, DTOs, serviços, controllers, frontend e testes.	Alterações atuais: criar commit/PR com autoria
Filipe de Carvalho Teotônio	Contribuição individual precisa ser confirmada pela equipe.	Indicar funcionalidade, requisito e arquivo desenvolvido.	Nenhum commit identificado no histórico analisado

Atendimento aos requisitos gerais

Requisito	Atendimento	Localização
Herança	Aluno e Professor herdam de Pessoa.	negocio.basica
Polimorfismo	exibirDados() possui implementações específicas.	Pessoa, Aluno e Professor
Encapsulamento	Estado privado e acesso controlado.	Todas as entidades
Associações	Professor-Turma, Turma-Disciplina, Matrícula e Nota.	Anotações JPA
Interfaces	Sete repositórios JpaRepository.	pacote dados
API e validação	Seis CRUDs, DTOs e ControllerAdvice.	pacote comunicacao
Testes	15 testes aprovados.	src/test/java

Procedimento de prestação de contas

Cada integrante deve possuir ao menos um commit identificável, com mensagem que cite a funcionalidade entregue. Quando houver trabalho conjunto, a descrição do PR deve indicar quem modelou, implementou, testou ou documentou. Capturas de execução podem complementar a evidência, mas não substituem o vínculo entre pessoa, requisito e artefato no Git.

Coordenação da equipe

Para a apresentação, a divisão pode acompanhar os tópicos do relatório: problema e arquitetura; modelo e orientação a objetos; API, frontend e demonstração; testes, dificuldades e conclusões. A equipe deve ajustar essa sugestão às contribuições reais para que todos participem ativamente do vídeo.

Pendência de prestação de contas

Antes da entrega, a equipe deve revisar a divisão acima, substituir descrições provisórias por contribuições reais e criar commits ou PRs que permitam localizar cada artefato. Essa etapa é necessária principalmente para Filipe e para as funcionalidades recentes atribuídas a Antonio.

6. Conclusões

O Sistema Escolar alcançou o objetivo de transformar um modelo de domínio em uma aplicação funcional, organizada e verificável. O projeto demonstra herança, polimorfismo, encapsulamento, associações e interfaces em um contexto coerente. A solução passou de um conjunto inicial de entidades e repositórios para uma arquitetura completa, com serviços, DTOs, controllers, tratamento global de erros, interface web e testes automatizados.

A implementação cobre o fluxo acadêmico essencial: cadastrar aluno, professor e disciplina; criar uma turma com docente responsável; matricular o aluno; e registrar notas. O uso de relacionamentos JPA mantém os objetos conectados, enquanto as chaves estrangeiras e restrições únicas protegem a consistência no PostgreSQL.

Resultados alcançados

- Seis recursos com operações CRUD por API REST.
- Interface web responsiva servida pelo Spring Boot.
- Sete entidades persistentes e sete interfaces de repositório.
- Validações de obrigatoriedade, formato, intervalo e integridade.
- Erros padronizados para dados inválidos, recursos ausentes e conflitos.
- Quinze testes aprovados e coleção Postman para demonstração.

Limitações atuais

- O site ainda não possui formulário de edição, embora a API ofereça PUT.
- A aplicação não implementa autenticação ou autorização por perfil.
- ddl-auto: create apaga os dados a cada reinicialização.
- As credenciais do banco permanecem no arquivo de configuração local.
- A nota não registra tipo de avaliação, unidade ou peso.
- O sistema ainda não possui paginação, frequência, relatórios ou documentação OpenAPI.

Possibilidades de continuidade

A próxima etapa deve introduzir migrações versionadas, variáveis de ambiente e perfis de configuração. Em seguida, a equipe pode adicionar autenticação, edição completa no frontend, filtros e paginação. No domínio, avaliações com peso, cálculo de média, frequência e situação final ampliariam o valor acadêmico. Documentação OpenAPI e testes integrados com banco isolado melhorariam a manutenção.

Aprendizados

O trabalho consolidou conhecimentos sobre modelagem orientada a objetos, separação de responsabilidades, mapeamento relacional, contratos HTTP, validação, testes e diagnóstico por logs. Também evidenciou que a qualidade técnica precisa ser acompanhada de documentação e rastreabilidade das contribuições.

Fonte das evidências

Código-fonte, README, diagrama de classes, relatórios de teste Maven, coleção Postman e histórico Git do repositório local analisado em setembro de 2026.